SQLITE

面试 100 题

从 SQL 使用到 B-Tree / Pager / WAL / Query Planner / FTS5 / JSONB

题目 → 考点 → 思路 → 标准回答 → 深挖追问 → 易错点 → 工程实践

100	10	3.53.4
系统化真题型	核心专题	当前稳定版基线

版本基线：SQLite 3.53.4（2026-07-24）

原创编排，学习结构参考“剑指 Offer”式逐题拆解，不复刻其文字或版式。

前言：这不是“背答案”的 SQLite 题库

SQLite 的面试难点并不在 SQL 语法，而在于它把数据库服务器中通常隐藏的很多机制——文件格式、锁、日志、页面缓存、查询规划、并发边界——直接暴露给应用工程师。一个候选人如果只会 CREATE TABLE、SELECT 和 CREATE INDEX，很容易在“为什么 WAL 还有 SQLITE_BUSY”“为什么有索引却全表扫描”“为什么 INTEGER PRIMARY KEY 和 INT PRIMARY KEY 不一样”这类追问中失分。

本书把 100 道题组织为十条主线：架构定位、类型与 ROWID、事务与并发、Journal/WAL、索引、Query Planner、存储引擎、运维完整性、高级能力、生产系统设计。每一题都按面试现场可复用的结构展开，目标不是记一句结论，而是建立能够从底层推出答案的心智模型。

建议学习顺序：第一遍只看“30 秒回答 + 核心结论”；第二遍精读“原理拆解”；第三遍关掉答案口述；第四遍针对 S 级题做白板推演。真正的高分答案应能把 SQL 层行为连接到 B-Tree、Pager、WAL 和操作系统文件锁。

> **版本提醒** 截至 2026-09-02，SQLite 最新稳定版为 3.53.4（发布于 2026-07-24）。3.53 系列包含 WAL-reset 数据损坏问题修复、ALTER TABLE 约束增强、REINDEX EXPRESSIONS 等变化。本书对版本敏感题会明确标注。

如何使用本书

- ★ 高频：常见于移动端、客户端、后端基础设施面试。
- ◆ 深挖：常用于区分“会用”与“懂实现”。
- ⚠ 易错：容易把 MySQL/PostgreSQL 经验错误迁移到 SQLite。
- V 版本敏感：回答前要确认 SQLite 版本与 journal mode。

十章路线图

章	主题	范围	题号
01	**架构与定位**	Embedded / Serverless / ACID / 使用边界	1-10
02	**类型、Schema 与 ROWID**	Affinity / STRICT / INTEGER PRIMARY KEY	11-20
03	**事务、隔离与并发**	DEFERRED / IMMEDIATE / Busy / Snapshot	21-30
04	**Journal、WAL 与持久性**	Rollback / WAL / Checkpoint / Crash	31-40
05	**索引设计**	联合 / 覆盖 / Partial / Expression	41-50
06	**Query Planner 与 SQL 优化**	EQP / ANALYZE / Join / OR / Skip-scan	51-60
07	**存储引擎与底层**	VDBE / B-Tree / Page / Pager / Overflow	61-70

08	**运维、备份与完整性**	VACUUM / PRAGMA / FK / Corruption	71-80
09	**高级能力**	FTS5 / JSONB / Generated / UPSERT	81-90
10	**生产场景与系统设计**	Lock / 批量写 / 亿级数据 / Offline-first	91-100

01

第一章 SQLite 架构与定位

先回答“SQLite 是什么、为什么存在、边界在哪里”。这一章决定后续所有并发和存储题是否有统一坐标。

本章目标 掌握本章后，应能在白板上解释关键机制，并能把“结论”推导到“为什么”。回答时优先先给结论，再给底层因果链。

题号	题目	难度/频率
001	**SQLite 是什么？与 MySQL/PostgreSQL 最大区别是什么？**	中 / 高
002	**SQLite 为什么不需要启动数据库服务？**	中 / 高
003	**SQLite 最适合哪些场景？**	中 / 高
004	**哪些场景不适合 SQLite？**	中 / 高
005	**SQLite 能保证 ACID 吗？**	中 / 高
006	**SQLite 是内存数据库还是磁盘数据库？**	中 / 高
007	**SQLite 数据库理论最大能有多大？**	中 / 高
008	**“单文件数据库”是否意味着永远只有一个文件？**	中 / 高
009	**为什么 SQLite 在移动端特别流行？**	中 / 高
010	**如何用一句话概括 SQLite 的设计哲学？**	中 / 高

题 001 SQLite 是什么？与 MySQL/PostgreSQL 最大区别是什么？

难度 中	频率 高	标签 架构 / Serverless	来源 Architecture

30 秒标准回答

> **结论** SQLite 是一个嵌入式、serverless 的关系数据库引擎。应用把 SQLite library 链接进进程，直接访问数据库文件；MySQL/PostgreSQL 通常由独立 server process 统一管理连接、缓存、锁和存储。

思路分析 / Know-Why

面试要把“没有服务器”讲成架构差异，而不是能力缺失。SQLite 仍然包含 SQL parser、query planner、VDBE、B-Tree、Pager、VFS 等完整数据库组件，只是这些组件运行在应用进程内。其优势是部署简单、进程内调用开销低、数据库天然随应用本地化；代价则是跨机器共享和高写并发不是它的主战场。真正的选择标准不是“数据量小不小”，而是数据是否属于单机、writer 冲突是否可控。

易错点

- ⚠ 把 serverless 理解成“没有数据库引擎”。
- ⚠ 仅用“SQLite 小、MySQL 大”做判断。

面试官可能继续追问

- → 为什么 SQLite 仍然能保证 ACID?
- → 什么场景应主动放弃 SQLite?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 002 SQLite 为什么不需要启动数据库服务?

难度 中	频率 高	标签 架构 / VDBE	来源 Architecture

30 秒标准回答

> **结论** 因为 SQL 解析、优化、执行、事务和文件 I/O 全部由应用进程内的 SQLite library 完成。

思路分析 / Know-Why

sqlite3_prepare_v2() 会把 SQL 编译成 VDBE bytecode；sqlite3_step() 驱动虚拟机执行。执行过程中通过 B-Tree 层访问表和索引，通过 Pager 管理 page cache、事务和日志，再经 VFS 调用操作系统文件接口。这条链路说明 SQLite 不是“文件读写工具”，而是完整数据库引擎的 library form。

代码 / 白板示例

```
SQL → Parser/Planner → VDBE → B-Tree → Pager → VFS → OS
```

易错点

- ⚠ 把 SQLite 当作“SQL 封装的文件 API”。

面试官可能继续追问

- → VDBE 与 JVM 类比是否合理?
- → Pager 为什么看不到 SQL 语义?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 003 SQLite 最适合哪些场景?

难度 中	频率 高	标签 选型	来源 When to use

30 秒标准回答

> **结论** 本地、单机、嵌入式、读多写少或写并发可控的场景：移动端、桌面端、IoT、edge、本地缓存、应用文件格式。

思路分析 / Know-Why

SQLite 的优势来自“数据与应用同机”：没有网络 round trip、无需单独部署服务、事务与 schema 能直接封装本地状态。很多单机 workload 即使达到 GB 甚至更大规模也可以很好工作。判断时应同时评估访问拓扑、写事务时长、备份与同步需求，而不是仅看行数。

易错点

- ⚠ 认为 SQLite 只能保存几 MB。

面试官可能继续追问

- → 1000 万行是否就该换 PostgreSQL?
- → 离线优先 App 为什么适合 SQLite?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 004 哪些场景不适合 SQLite?

难度 中	频率 高	标签 选型 / 并发	来源 WAL

30 秒标准回答

> **结论** 多机共享同一数据库文件、持续高写并发、需要服务端权限治理/连接池/横向扩展的场景通常不适合。

思路分析 / Know-Why

SQLite 的并发瓶颈不是 reader，而是同一数据库最终只能有一个 writer。若业务要求很多机器通过网络文件系统并发访问同一个数据库，文件锁语义和 WAL 的共享内存约束都会变成风险。此时 client/server DB 更符合架构。

易错点

- ⚠ 把 WAL 当成“多 writer 并行开关”。
- ⚠ 在普通 NFS 上共享 WAL 数据库。

面试官可能继续追问

- → WAL 为什么仍只有一个 writer?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 005 SQLite 能保证 ACID 吗?

难度 中	频率 高	标签 ACID / Pager	来源 Locking

30 秒标准回答

> **结论** 能。事务原子性、隔离与持久性主要由 Pager + journal/WAL + 文件锁协同保证。

思路分析 / Know-Why

Rollback journal 模式采用 before-image：覆盖数据库页前先保存旧页；崩溃后可利用 hot journal 回滚。WAL 模式则把 after-image 追加到 WAL，提交通过 commit frame 标记，随后 checkpoint 回写主库。Consistency 则同时依赖 SQLite 的约束机制和应用维护的业务不变量。

易错点

- ⚠ 把 ACID 等同于 synchronous=FULL。

面试官可能继续追问

- → NORMAL 与 FULL 对 D 的影响是什么?
- → hot journal 是什么?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 006 SQLite 是内存数据库还是磁盘数据库?

难度 中	频率 高	标签 基础	来源 URI

30 秒标准回答

> **结论** 两者都支持。普通路径创建文件数据库，:memory: 创建仅存在于 connection 生命周期内的内存库。

思路分析 / Know-Why

嵌入式描述的是部署模型，不等于存储介质。还要注意多个独立 :memory: connection 默认不是同一个数据库；共享内存数据库需要特定 URI/shared cache 语义，工程上应谨慎使用。

代码 / 白板示例

```
sqlite3_open(":memory:", &db);
```

易错点

- ⚠ “embedded = in-memory”。

面试官可能继续追问

- → 多 connection 如何共享一个内存库?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 007 SQLite 数据库理论最大能有多大?

难度 中	频率 高	标签 容量 / Page	来源 File format

30 秒标准回答

> **结论** 理论上取决于 page size × 最大 page count；在最大 page size 下可到数百 TB 量级。

思路分析 / Know-Why

数据库的物理单位是 page，page size 在现代 SQLite 中通常为 4096，允许 512 到 65536。理论上限很高，但工程上往往先受文件系统、备份窗口、VACUUM 空间、缓存命中和查询设计限制。面试回答应避免把“理论文件上限”误当成“适合处理的业务规模”。

易错点

- ⚠ 只背一个 TB 数字，不解释 page 模型。

面试官可能继续追问

- → page size 是否越大越好?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 008 “单文件数据库”是否意味着永远只有一个文件?

难度 中	频率 高	标签 文件 / WAL	来源 File format

30 秒标准回答

> **结论** 不是。主库通常是一个文件，但事务期间可能出现 -journal、-wal、-shm 等辅助文件。

思路分析 / Know-Why

Rollback 模式可能创建 journal；WAL 模式通常存在 database-wal 与 database-shm。备份、复制、删除数据库时如果忽视这些辅助状态，就可能得到不一致副本。生产回答要把“单文件”理解为应用数据模型，而不是运行时绝对只有一个 inode。

易错点

- ⚠ 在线直接 cp 主 db 文件而忽略 WAL。

面试官可能继续追问

- → 如何做一致性在线备份?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 009 为什么 SQLite 在移动端特别流行?

难度 中	频率 高	标签 移动端	来源 WAL

30 秒标准回答

> **结论** 因为它把事务、SQL、索引、崩溃恢复和本地文件持久化放在一个零部署 library 中，非常符合单设备本地状态模型。

思路分析 / Know-Why

移动端经常是“UI 多读 + 后台少量写 + 离线可用”。WAL 能改善读写重叠，单文件方便迁移与备份，SQL 又能支持复杂筛选与排序。真正需要额外设计的是生命周期、schema migration、同步冲突和长事务。

易错点

- ⚠ 说“因为它轻量”而不解释 workload 匹配。

面试官可能继续追问

- → Android 中为什么仍可能出现 database is locked?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 010 如何用一句话概括 SQLite 的设计哲学?

难度 中	频率 高	标签 架构总览	来源 Architecture

30 秒标准回答

> **结论** 把完整数据库引擎嵌入应用，让数据库成为应用的一部分，而不是远程服务。

思路分析 / Know-Why

这句话背后对应一个重要面试框架：SQL 层的每个行为都能沿着 Planner → VDBE → B-Tree → Pager → VFS 追到底层。后续索引、事务、WAL 和 corruption 题都可以从这条链路推导。

代码 / 白板示例

```
SQL → Planner → VDBE → B-Tree → Pager → VFS
```

易错点

- ⚠ 只把 SQLite 当“轻量 MySQL”。

面试官可能继续追问

- → 这一哲学带来的最大并发代价是什么?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

02

第二章 类型、Schema 与 ROWID

SQLite 最反直觉的部分：动态类型、Affinity、STRICT、ROWID 与 INTEGER PRIMARY KEY。

本章目标 掌握本章后，应能在白板上解释关键机制，并能把“结论”推导到“为什么”。回答时优先先给结论，再给底层因果链。

题号	题目	难度/频率
011	**SQLite 真正有哪些数据类型?**	中 / 高
012	**什么是 Type Affinity?**	中 / 高
013	**VARCHAR(10) 是否最多存 10 个字符?**	中 / 高
014	**SQLite 有 BOOLEAN 类型吗?**	中 / 高
015	**SQLite 如何保存日期时间?**	中 / 高
016	**STRICT TABLE 解决了什么问题?**	中 / 高
017	**STRICT TABLE 中 ANY 有什么特殊语义?**	中 / 高
018	**什么是 ROWID?**	中 / 高
019	**INTEGER PRIMARY KEY 和 INT PRIMARY KEY 一样吗?**	中 / 高
020	**INTEGER PRIMARY KEY AUTOINCREMENT 与普通 INTEGER PRIMARY KEY 的区别?**	中 / 高

题 011 SQLite 真正有哪些数据类型?

难度 中	频率 高	标签 类型系统	来源 Datatypes

30 秒标准回答

> **结论** 值有五种 storage class：NULL、INTEGER、REAL、TEXT、BLOB。

思路分析 / Know-Why

SQLite 的动态类型系统把类型主要附着在“值”上，而不是容器上。列声明通过 type affinity 影响转换倾向；这与 MySQL/PostgreSQL 的 rigid typing 思路不同。面试时先讲 storage class，再讲 affinity，最后补 STRICT table，层次最清楚。

易错点

- ⚠ 直接回答 INT/VARCHAR/DOUBLE。

面试官可能继续追问

- → typeof() 返回的是什么?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 012 什么是 Type Affinity?

难度 中	频率 高	标签 Affinity	来源 Datatypes

30 秒标准回答

> **结论** 列对输入值的“推荐存储类型倾向”，不是普通表中的绝对类型约束。

思路分析 / Know-Why

SQLite 根据声明类型名称映射到 INTEGER/TEXT/REAL/BLOB/NUMERIC affinity。写入时会尝试无损转换，例如数字字符串进入 INTEGER affinity 列可能被保存成整数；无法适当转换时，普通表仍可保存原值。比较、排序和索引匹配也会受 affinity 与 coercion 影响，因此它不仅是写入问题。

代码 / 白板示例

```
CREATE TABLE t(a INTEGER);
INSERT INTO t VALUES('123'); -- 通常存成整数 123
```

易错点

- ⚠ 认为 affinity = strict type。

面试官可能继续追问

- → NUMERIC 与 INTEGER affinity 的差异在哪里?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 013 VARCHAR(10) 是否最多存 10 个字符?

难度 中	频率 高	标签 类型陷阱	来源 Datatypes

30 秒标准回答

> **结论** 不是。SQLite 不把括号中的长度当作自动 CHECK 约束。

思路分析 / Know-Why

VARCHAR(10) 主要参与声明类型到 affinity 的映射；它不会像很多服务器数据库一样强制长度 <=10。若业务需要硬约束，应显式 CHECK(length(name)<=10)，或在应用层与数据库层同时验证。

代码 / 白板示例

```
CREATE TABLE user(
  name TEXT CHECK(length(name) <= 10)
);
```

易错点

- ⚠ 把其他数据库的 VARCHAR 长度语义直接迁移过来。

面试官可能继续追问

- → STRICT table 会不会自动让 VARCHAR(10) 生效?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 014 SQLite 有 BOOLEAN 类型吗?

难度 中	频率 高	标签 类型	来源 Datatypes

30 秒标准回答

> **结论** 没有独立 BOOLEAN storage class；TRUE/FALSE 通常映射为整数 1/0。

思路分析 / Know-Why

Schema 中写 BOOLEAN 主要得到特定 affinity 行为，并不会创造新的物理类型。生产 schema 如果要限制只能 0/1，可写 CHECK(flag IN (0,1))，STRICT 也不会引入单独 boolean storage class。

易错点

- ⚠ 认为 BOOLEAN 占 1-bit。

面试官可能继续追问

- → NULL 是否应该允许用于三态逻辑?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 015 SQLite 如何保存日期时间?

难度 中	频率 高	标签 时间	来源 Date functions

30 秒标准回答

> **结论** 常见用 ISO-8601 TEXT、Unix timestamp INTEGER、Julian day REAL。

思路分析 / Know-Why

SQLite 没有独立 DATETIME storage class。设计应先确定：是否需要时区、排序、范围查询、跨语言序列化。ISO-8601 的规范文本在统一格式下具备良好可读性与字典序；Unix timestamp 适合数值比较；最危险的是同一列混用多种表示。

易错点

- ⚠ 在同一列混存本地时间文本和 Unix 秒。

面试官可能继续追问

- → 如何索引按天查询?
- → UTC 与本地时区应在哪一层处理?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 016 STRICT TABLE 解决了什么问题?

难度 中	频率 高	标签 STRICT / 版本敏感	来源 STRICT

30 秒标准回答

> **结论** 为单张表启用更严格的类型规则，无法无损转换到声明类型时返回 SQLITE_CONSTRAINT_DATATYPE。

思路分析 / Know-Why

STRICT 从 3.37.0 引入。STRICT 表每列必须声明允许的类型名（INT、INTEGER、REAL、TEXT、BLOB、ANY 等当前集合），但它并没有改变 SQLite 的磁盘格式和大部分 SQL 能力。它解决的是“灵活类型导致错误数据悄悄进入表”的工程痛点。

代码 / 白板示例

```
CREATE TABLE account(
  id INTEGER PRIMARY KEY,
  balance REAL NOT NULL
) STRICT;
```

易错点

- ⚠ 认为 STRICT 让 SQLite 变成 PostgreSQL 类型系统。

面试官可能继续追问

- → STRICT 中 ANY 为什么存在?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 017 STRICT TABLE 中 ANY 有什么特殊语义?

难度 中	频率 高	标签 STRICT	来源 STRICT

30 秒标准回答

> **结论** 允许任意类型，并尽量原样保留输入值，而不是像普通 affinity 那样积极数值化。

思路分析 / Know-Why

ANY 是 SQLite 为保留动态数据能力而提供的逃生口。它适合确实需要异构 payload 的列，但如果业务字段本来应该稳定，就不要用 ANY 逃避 schema 设计。

易错点

- ⚠ 把 ANY 当成“没有 schema 就更灵活”。

面试官可能继续追问

- → ANY 与 BLOB 有什么区别?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 018 什么是 ROWID?

难度 中	频率 高	标签 ROWID / B-Tree	来源 File format

30 秒标准回答

> **结论** 普通 rowid table 隐含一个 64-bit signed integer key，可通过 rowid/_rowid_/oid 访问（除非名称被显式列遮蔽）。

思路分析 / Know-Why

在普通表的 table b-tree 中，ROWID 就是每条 entry 的 key；row payload 作为 value。这就是为什么 INTEGER PRIMARY KEY 作为 ROWID alias 的访问路径非常高效。理解 ROWID 是理解表组织、二级索引回表、AUTOINCREMENT 和 WITHOUT ROWID 的核心。

易错点

- ⚠ 把 ROWID 当成普通隐藏列。

面试官可能继续追问

- → 二级索引叶子为什么要保存 rowid?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 019 INTEGER PRIMARY KEY 和 INT PRIMARY KEY 一样吗?

难度 中	频率 高	标签 ROWID / PK	来源 STRICT

30 秒标准回答

> **结论** 不一样。普通 rowid table 中，声明精确为 INTEGER PRIMARY KEY 时该列成为 ROWID alias；INT PRIMARY KEY 通常不会。

思路分析 / Know-Why

这是 SQLite 最经典的语法-存储耦合题。INTEGER PRIMARY KEY 的值直接来自 table b-tree key，因此不用再维护一份独立整数主键索引。STRICT 表中这一规则仍成立：INTEGER PRIMARY KEY 是 rowid alias，而 INT PRIMARY KEY 不是。

代码 / 白板示例

```
CREATE TABLE a(id INTEGER PRIMARY KEY, v TEXT); -- id = rowid
CREATE TABLE b(id INT PRIMARY KEY, v TEXT);     -- id 通常不是 rowid alias
```

易错点

- ⚠ 认为 INT 是 INTEGER 的完全同义词。

面试官可能继续追问

- → DESC 主键有没有历史例外?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 020 INTEGER PRIMARY KEY AUTOINCREMENT 与普通 INTEGER PRIMARY KEY 的区别?

难度 中	频率 高	标签 ROWID / AUTOINCREMENT	来源 Autoincrement

30 秒标准回答

> **结论** 普通 INTEGER PRIMARY KEY 已经自动分配 ROWID；AUTOINCREMENT 额外保证自动生成的 ROWID 不复用历史上已提交过的 ROWID，并带来额外开销。

思路分析 / Know-Why

默认算法通常使用当前最大 ROWID+1，但某些删除场景可能复用旧值。AUTOINCREMENT 通过 sqlite_sequence 追踪历史最大值，适合“ID 永不复用”确有业务语义的场景；不要为了“自动增长”机械加它。它保证单调增加，不保证连续无缺口。

易错点

- ⚠ 认为不加 AUTOINCREMENT 就不会自动生成 ID。
- ⚠ 认为 AUTOINCREMENT 能保证无 gap。

面试官可能继续追问

- → rollback 的 ROWID 是否算“历史已使用”？

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

03

第三章 事务、隔离与并发

从 implicit transaction 到 DEFERRED/IMMEDIATE，再到 Busy 与 snapshot。

本章目标 掌握本章后，应能在白板上解释关键机制，并能把“结论”推导到“为什么”。回答时优先先给结论，再给底层因果链。

题号	题目	难度/频率
021	**不写 BEGIN，一条 INSERT 有事务吗?**	中 / 高
022	**BEGIN DEFERRED 是什么?**	中 / 高
023	**BEGIN IMMEDIATE 与 DEFERRED 有何区别?**	中 / 高
024	**BEGIN EXCLUSIVE 与 IMMEDIATE 有何区别?**	中 / 高
025	**SQLite 支持并发读吗?**	中 / 高
026	**SQLite 同一数据库能同时有两个 Writer 吗?**	中 / 高
027	**Read transaction 升级为 Write transaction 为什么会失败?**	中 / 高
028	**busy_timeout 是什么？能解决所有锁问题吗?**	中 / 高
029	**SQLite 支持嵌套事务吗?**	中 / 高
030	**PRAGMA read_uncommitted=ON 就一定能脏读吗?**	中 / 高

题 021 不写 BEGIN，一条 INSERT 有事务吗?

难度 中	频率 高	标签 事务	来源 Transaction

30 秒标准回答

> **结论** 有。SQLite 会自动开启 implicit transaction，并在最后一个活动 statement 完成后自动提交。

思路分析 / Know-Why

事务是数据库修改的基本单位。即使应用没有显式 BEGIN，写语句仍在事务内执行。需要注意“statement 完成”与 cursor/statement 生命周期相关：prepared statement 应 reset/finalize，incremental BLOB handle 也会影响事务结束。

易错点

- ⚠ “没有 BEGIN 就没有事务”。

面试官可能继续追问

- → 为什么没 finalize statement 会让事务比想象中更久?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 022 BEGIN DEFERRED 是什么?

难度 中	频率 高	标签 事务 / 锁	来源 Transaction

30 秒标准回答

> **结论** 默认事务模式。BEGIN 本身只关闭 auto-commit，真正读/写事务在首次访问时形成。

思路分析 / Know-Why

若第一条语句是 SELECT，就先进入 read transaction；后续写操作需要升级成 write transaction，如果别的连接已在修改数据库，升级可能 SQLITE_BUSY。DEFERRED 的优点是晚抢资源，缺点是 read-modify-write 场景可能在做完一堆读取后才发现无法写。

代码 / 白板示例

```
BEGIN DEFERRED;
SELECT ...;
UPDATE ...;
COMMIT;
```

易错点

- ⚠ 认为 BEGIN DEFERRED 立即获取写锁。

面试官可能继续追问

- → 什么时候应该改用 IMMEDIATE?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 023 BEGIN IMMEDIATE 与 DEFERRED 有何区别?

难度 中	频率 高	标签 事务 / Busy	来源 Transaction

30 秒标准回答

> **结论** IMMEDIATE 在 BEGIN 时就尝试启动 write transaction，已有 writer 时可立即得到 SQLITE_BUSY。

思路分析 / Know-Why

它适合确定“这一事务最终必须写”的 read-modify-write 流程：提前竞争 writer，避免基于旧 snapshot 做完复杂计算后升级失败。代价是更早占用 writer 槽位，所以不要把所有事务都无脑改成 IMMEDIATE。

易错点

- ⚠ 把 IMMEDIATE 理解为“立刻提交”。

面试官可能继续追问

- → busy_timeout 对 BEGIN IMMEDIATE 怎么生效?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 024 BEGIN EXCLUSIVE 与 IMMEDIATE 有何区别?

难度 中	频率 高	标签 事务 / 版本语义	来源 Transaction

30 秒标准回答

> **结论** 在 WAL mode 中二者基本相同；在其他 journal mode 中 EXCLUSIVE 还会阻止其他连接读取。

思路分析 / Know-Why

这是一道典型“必须先问 journal_mode”的题。若候选人脱离日志模式直接说 EXCLUSIVE 一定阻塞 reader，就是答得不完整。面试中应先建立上下文：rollback journal 还是 WAL。

易错点

- ⚠ 忽略 journal mode。

面试官可能继续追问

- → 为什么 WAL 中 EXCLUSIVE 不再排斥 reader?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 025 SQLite 支持并发读吗?

难度 中	频率 高	标签 并发	来源 Locking/WAL

30 秒标准回答

> **结论** 支持多个 reader；读写是否互相阻塞取决于 journal mode。

思路分析 / Know-Why

Rollback journal 中多个 SHARED reader 可以并发，但 writer 进入提交阶段需要更强锁。WAL 中 reader 固定在自己的 snapshot/end mark，writer 追加 WAL，因此读写能更好重叠。并发读能力通常不是 SQLite 的主要瓶颈。

易错点

- ⚠ “SQLite 只能单线程访问”。

面试官可能继续追问

- → 同进程线程安全与数据库并发是同一件事吗?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 026 SQLite 同一数据库能同时有两个 Writer 吗?

难度 中	频率 高	标签 并发 / WAL	来源 WAL

30 秒标准回答

> **结论** 不能真正并行写；WAL 也只有一个 writer。

思路分析 / Know-Why

WAL 解决的是 reader 与 writer 冲突，不是 writer 与 writer 并行。所有写入最终要形成单一序列并维护 page/WAL 一致性；第二个 writer 必须等待或收到 SQLITE_BUSY。工程上因此常采用短写事务、批处理、writer queue 或受控 writer pool。

易错点

- ⚠ “开启 WAL 就支持多 writer”。

面试官可能继续追问

- → 为什么专用写线程可能比 100 个写线程更快?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 027 Read transaction 升级为 Write transaction 为什么会失败?

难度 中	频率 高	标签 Snapshot / Busy	来源 Transaction

30 秒标准回答

> **结论** 因为 DEFERRED 事务先持有一个历史 snapshot；如果其他连接已修改/正在修改数据库，升级写事务可能无法成立并返回 SQLITE_BUSY。

思路分析 / Know-Why

尤其在 WAL 中，读事务看到的 snapshot 不会随着其他连接 commit 自动前移。某些场景会出现 SQLITE_BUSY_SNAPSHOT：当前连接若写入，会基于已经过时的历史视图分叉。若业务语义要求“读到的状态就是随后修改的依据”，从 BEGIN IMMEDIATE 开始通常更可预测。

易错点

- ⚠ 认为 busy_timeout 一定能把 snapshot 升级问题等到成功。

面试官可能继续追问

- → SQLITE_BUSY 与 SQLITE_BUSY_SNAPSHOT 区别?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 028 busy_timeout 是什么？能解决所有锁问题吗？

难度 中	频率 高	标签 Busy / 工程	来源 Pragma

30 秒标准回答

> **结论** 它让 SQLite 遇到可等待的锁冲突时在指定时间内重试，而不是立即返回 SQLITE_BUSY；它不是并发架构修复器。

思路分析 / Know-Why

如果问题来自长事务、网络 I/O 放在事务中、连接忘记提交、长期 reader 阻塞 checkpoint，单纯把 timeout 从 5s 调到 60s 只会把问题变成更长延迟。正确流程是先定位持锁时间，再决定 timeout 作为毛刺缓冲。

代码 / 白板示例

```
PRAGMA busy_timeout = 5000;
```

易错点

- ⚠ 用超长 timeout 掩盖长事务。

面试官可能继续追问

- → 什么时候应该应用层指数退避?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 029 SQLite 支持嵌套事务吗?

难度 中	频率 高	标签 SAVEPOINT	来源 Transaction

30 秒标准回答

> **结论** BEGIN...COMMIT 事务不能直接嵌套；应使用 SAVEPOINT / ROLLBACK TO / RELEASE。

思路分析 / Know-Why

SAVEPOINT 提供局部回滚边界，特别适合 library 内部操作：外层调用者可能已经在事务里，内部模块不能再 BEGIN，但可以建立 savepoint。ROLLBACK TO 会回到保存点而不自动结束最外层事务。

代码 / 白板示例

```
SAVEPOINT s1;
...
ROLLBACK TO s1;
RELEASE s1;
```

易错点

- ⚠ 在已 BEGIN 的事务里再次 BEGIN。

面试官可能继续追问

- → RELEASE 是否等价于最外层 COMMIT?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 030 PRAGMA read_uncommitted=ON 就一定能脏读吗?

难度 中	频率 高	标签 隔离级别	来源 Isolation

30 秒标准回答

> **结论** 不是。普通独立 connection 仍保持隔离；只有 shared-cache 等特殊组合下才可能看到未提交变化。

思路分析 / Know-Why

SQLite 的常规隔离通过文件/页面视图实现。shared-cache 是历史机制，官方对多数场景不推荐使用，很多过去为提高并发而使用 shared-cache 的需求如今可用 WAL 更好解决。面试应避免把 SQL Server/MySQL 中 read uncommitted 的直觉直接移植过来。

易错点

- ⚠ 认为一条 PRAGMA 就把全库变成脏读。

面试官可能继续追问

- → WAL 提供的 snapshot isolation 如何理解?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

04

第四章 Rollback Journal、WAL 与持久性

理解 WAL 不是背 PRAGMA，而是理解 page version、checkpoint 与 crash recovery。

本章目标 掌握本章后，应能在白板上解释关键机制，并能把“结论”推导到“为什么”。回答时优先先给结论，再给底层因果链。

题号	题目	难度/频率
031	**Rollback Journal 的基本原理是什么?**	中 / 高
032	**WAL 的基本原理是什么?**	中 / 高
033	**WAL 为什么允许 Reader 与 Writer 并发?**	中 / 高
034	**WAL 是否意味着完全没有锁?**	中 / 高
035	**为什么 WAL 不适合普通网络文件系统?**	中 / 高
036	**Checkpoint 是什么?**	中 / 高
037	**为什么长期 Reader 会导致 WAL 越来越大?**	中 / 高
038	**PRAGMA synchronous 控制什么?**	中 / 高
039	**什么是 Hot Journal?**	中 / 高
040	**2026 年为什么 SQLite 3.53 系列值得关注?**	中 / 高

题 031 Rollback Journal 的基本原理是什么?

难度 中	频率 高	标签 Journal / ACID	来源 Atomic commit

30 秒标准回答

> **结论** 覆盖数据库页前先把旧页写入 journal；事务失败或崩溃时用旧页回滚。

思路分析 / Know-Why

这是 before-image 方案。提交过程需要保证 journal 与数据库写入顺序和 sync 屏障，使崩溃发生在任意阶段都能判断该回滚还是保留。理解这一点后就能推导 hot journal、DELETE/TRUNCATE/PERSIST journal_mode 等问题。

易错点

- ⚠ 把 rollback journal 说成 redo log。

面试官可能继续追问

- → 为什么 journal 与主库不能随意分别复制?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 032 WAL 的基本原理是什么?

难度 中	频率 高	标签 WAL	来源 WAL/File format

30 秒标准回答

> **结论** 主库暂时保留旧页，新版本 page frame 追加写入 WAL；事务以 commit marker 提交，之后 checkpoint 再回写主库。

思路分析 / Know-Why

这是 after-image/append 思路。Reader 结合主库与 WAL 获取自己 snapshot 所需的最新页；writer 多数时间只需顺序追加 WAL，从而避免直接覆盖 reader 正在使用的 page。WAL 的性能优势来自减少某些随机覆盖与读写阻塞，但不是“所有 workload 都更快”。

易错点

- ⚠ 认为 commit 时必须先把 WAL 全量 checkpoint。

面试官可能继续追问

- → WAL commit 与 checkpoint 是不是同一件事?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 033 WAL 为什么允许 Reader 与 Writer 并发?

难度 中	频率 高	标签 WAL / Snapshot	来源 WAL

30 秒标准回答

> **结论** Reader 固定自己的 end mark/snapshot，writer 只在 WAL 尾部追加新 frames，不覆盖 reader 所依赖的主库视图。

思路分析 / Know-Why

读取某页时，SQLite 先借助 wal-index 找 end mark 之前的最新 frame；没有才回到主库。因此 writer 的新提交可以在 reader 继续读旧 snapshot 时存在。这个机制也解释了长期 reader 为什么会限制 checkpoint 前进。

易错点

- ⚠ 把 reader/writer 并发误解成两个 writer。

面试官可能继续追问

- → wal-index 的作用是什么?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 034 WAL 是否意味着完全没有锁?

难度 中	频率 高	标签 WAL / 锁	来源 WAL

30 秒标准回答

> **结论** 不是。WAL 仍依赖数据库/WAL/SHM 锁来协调 reader、writer 和 checkpoint；同一时刻仍只有一个 writer。

思路分析 / Know-Why

WAL 的共享内存 wal-index 还要记录 reader marks、frame 映射等状态。如果文件系统锁失效或应用错误操作 WAL/SHM 文件，依然可能造成故障甚至 corruption。正确表述是“锁冲突模式发生变化”，不是“没有锁”。

易错点

- ⚠ WAL=no lock。

面试官可能继续追问

- → WAL 下普通 SELECT 为什么偶尔也可能遇到 BUSY?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 035 为什么 WAL 不适合普通网络文件系统?

难度 中	频率 高	标签 WAL / 文件系统	来源 WAL/Corruption

30 秒标准回答

> **结论** wal-index 依赖同机共享内存与可靠文件锁；网络文件系统通常无法满足 SQLite WAL 所需语义。

思路分析 / Know-Why

官方文档明确指出，WAL 要求所有 reader 在同一机器，因为 wal-index 使用 shared memory。再叠加 NFS 等文件系统可能存在 advisory lock 实现问题，跨机器共享同一 WAL 数据库风险高。若需要多机访问，应改用真正 client/server DB。

易错点

- ⚠ 通过 NFS 共享 sqlite.db-wal。

面试官可能继续追问

- → rollback journal 就一定适合 NFS 吗?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 036 Checkpoint 是什么?

难度 中	频率 高	标签 WAL / Checkpoint	来源 WAL

30 秒标准回答

> **结论** 把 WAL 中已提交的有效 page frames 复制回主数据库。

思路分析 / Know-Why

Checkpoint 可与 reader 部分并发，但不能越过仍被某个 reader snapshot 依赖的 WAL 点。完整 checkpoint 后，在没有连接仍引用旧 WAL 状态时，后续写事务可以 reset/reuse WAL。checkpoint 策略影响写延迟、WAL 大小与读性能。

易错点

- ⚠ 把 checkpoint 当 commit。

面试官可能继续追问

- → PASSIVE/FULL/RESTART/TRUNCATE checkpoint 模式差异?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 037 为什么长期 Reader 会导致 WAL 越来越大?

难度 中	频率 高	标签 Checkpoint starvation	来源 WAL

30 秒标准回答

> **结论** 因为 checkpoint 不能覆盖仍被老 snapshot 需要的页面，只能停在该 reader 的 end mark 前；writer 继续追加导致 WAL 增长。

思路分析 / Know-Why

这通常被称为 checkpoint starvation。常见根因是游标/statement 长时间不结束、业务把分页或网络操作包在读事务里、连接泄漏。解决策略首先是缩短读事务生命周期，其次才是调整 checkpoint 配置。

易错点

- ⚠ 看到 WAL 大就直接 rm -wal。

面试官可能继续追问

- → 如何定位哪个连接长期持有 reader?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 038 PRAGMA synchronous 控制什么?

难度 中	频率 高	标签 Durability	来源 Pragma

30 秒标准回答

> **结论** 控制 SQLite 在关键阶段使用 VFS xSync/同步屏障的积极程度，本质是持久性与性能的权衡。

思路分析 / Know-Why

OFF/NORMAL/FULL/EXTRA 的精确风险取决于 journal mode。它不决定“有没有事务”，而影响电源故障、OS crash 等场景下已经返回成功的提交能否可靠存活。面试高分答案要避免给出跨 journal mode 的绝对化结论。

易错点

- ⚠ synchronous=OFF 就等于没有 ACID。

面试官可能继续追问

- → WAL+NORMAL 与 rollback+FULL 的风险模型有何不同?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 039 什么是 Hot Journal?

难度 中	频率 高	标签 Crash recovery	来源 Locking

30 秒标准回答

> **结论** rollback journal 中断后留下的、表明主库可能含有未完整提交修改的恢复日志。

思路分析 / Know-Why

下次打开数据库时 SQLite 会判断 journal 是否为 hot，并在正常访问前恢复旧页面，使主库回到一致状态。绝对不要把“程序崩溃留下 journal”直接当垃圾文件删除。

易错点

- ⚠ 手工删除看起来“多余”的 -journal。

面试官可能继续追问

- → 如何判断 journal 是否 hot?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 040 2026 年为什么 SQLite 3.53 系列值得关注?

难度 中	频率 高	标签 版本 / 3.53	来源 Release 3.53.4

30 秒标准回答

> **结论** 3.53 系列修复了 WAL-reset corruption bug，并加入 ALTER TABLE 约束增强、REINDEX EXPRESSIONS、自愈式索引等改进；截至 2026-09-02 稳定版是 3.53.4。

思路分析 / Know-Why

这是版本敏感题。面试时若项目使用系统自带 SQLite，不能假设它等于 sqlite.org 最新版；Android/iOS/Python/浏览器可能捆绑不同版本。正确工程动作是运行时查询 sqlite_version()，再决定特性与规避方案。

代码 / 白板示例

```
SELECT sqlite_version();
```

易错点

- ⚠ 只看开发机 sqlite3 CLI 版本。

面试官可能继续追问

- → 如何做 feature gate?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

05

第五章 索引设计

从 B-Tree 顺序推导联合索引、覆盖索引、Partial 与 Expression Index。

本章目标 掌握本章后，应能在白板上解释关键机制，并能把“结论”推导到“为什么”。回答时优先先给结论，再给底层因果链。

题号	题目	难度/频率
041	**SQLite 普通索引底层是什么?**	中 / 高
042	**联合索引 (a,b,c) 能有效服务哪些条件?**	中 / 高
043	**什么是 Covering Index?**	中 / 高
044	**有 (a,b,c) 后还需要单列索引 (a) 吗?**	中 / 高
045	**什么是 Partial Index?**	中 / 高
046	**如何用 Partial UNIQUE Index 保证“每个用户最多一条 active 记录”?**	中 / 高
047	**什么是 Expression Index?**	中 / 高
048	**为什么 INDEX ON (x+y)，查询 WHERE y+x=10 可能不用索引?**	中 / 高
049	**为什么 Expression Index 只能使用 deterministic function?**	中 / 高
050	**为什么“有索引”反而可能比 Full Scan 慢?**	中 / 高

题 041 SQLite 普通索引底层是什么?

难度 中	频率 高	标签 索引 / B-Tree	来源 File format

30 秒标准回答

> **结论** 主要是 B-Tree（更准确说 index b-tree）；查询定位通常是 O(log N) 级树搜索。

思路分析 / Know-Why

普通 rowid table 有 table b-tree，二级索引各自是 index b-tree。索引 key 是索引列再附带 row key，因此能唯一定位到表行。理解“索引不是哈希表”才能解释范围查询、排序、前缀和覆盖索引。

易错点

- ⚠ 把数据库 index 一律说成 B+Tree 而不结合 SQLite 文件格式术语。

面试官可能继续追问

- → SQLite 的 table b-tree 与 index b-tree 有何差异?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 042 联合索引 (a,b,c) 能有效服务哪些条件?

难度 中	频率 高	标签 联合索引	来源 Optimizer overview

30 秒标准回答

> **结论** 通常先利用从最左开始的等值/IN/IS 约束，随后最右端可利用范围；中间出现缺口或范围后，后续列通常不能继续用于缩小搜索区间。

思路分析 / Know-Why

例如 a=? AND b=? AND c>? 非常理想；a=? AND c=? 缺少 b 时，一般无法直接按普通前缀规则使用 c 缩小同一 seek（但 planner 还有 skip-scan 等高级优化）。联合索引设计应同时考虑过滤、排序与返回列，而不是只统计 WHERE 中有哪些字段。

代码 / 白板示例

```
CREATE INDEX idx ON t(a,b,c);
```

易错点

- ⚠ 机械背“最左前缀”但不会分析范围条件。

面试官可能继续追问

- → a IN (...) AND b=? 如何使用?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 043 什么是 Covering Index?

难度 中	频率 高	标签 Covering Index	来源 Query planner

30 秒标准回答

> **结论** 索引包含查询所需的过滤列和输出列，使 SQLite 不必再回到原 table b-tree 取数据。

思路分析 / Know-Why

普通二级索引查询常经历：index seek → rowid → table seek。若 SELECT 所需列都在 index entry 中，就能减少第二次树查找和随机页访问。代价是索引更宽、写放大更大、占用更多 cache，因此要给真正热点查询使用。

代码 / 白板示例

```
CREATE INDEX idx_feed ON article(user_id,status,created_at,title);
```

易错点

- ⚠ 把所有列都塞进索引追求 covering。

面试官可能继续追问

- → 为什么 covering 只是常数因子优化?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 044 有 (a,b,c) 后还需要单列索引 (a) 吗?

难度 中	频率 高	标签 索引治理	来源 Query planner

30 秒标准回答

> **结论** 多数情况下不需要，因为联合索引已能按前缀 a 查找；保留冗余前缀索引会增加空间和写维护。

思路分析 / Know-Why

但“多数情况下”不是数学定律：索引宽度、cache、统计信息、特定排序/覆盖需求都可能使更窄索引有价值。正确做法是用真实 query plan 和 benchmark 评估，而不是教条删除。

易错点

- ⚠ 绝对化说“永远不需要”。

面试官可能继续追问

- → 如何通过 PRAGMA index_list/index_xinfo 审计冗余索引?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 045 什么是 Partial Index?

难度 中	频率 高	标签 Partial Index	来源 Partial index

30 秒标准回答

> **结论** 只为满足 CREATE INDEX ... WHERE 条件的行建立索引。

思路分析 / Know-Why

如果热点业务只查询 active、未删除、待同步等少量行，partial index 能显著缩小 B-Tree，降低写维护成本和缓存压力。例如 deleted=0 占 5% 时，索引只保存这 5%。planner 只有在能证明查询 WHERE 蕴含 partial predicate 时才会安全使用它。

代码 / 白板示例

```
CREATE INDEX idx_active_user ON orders(user_id) WHERE deleted=0;
```

易错点

- ⚠ partial index 条件与查询条件写得不兼容导致不使用。

面试官可能继续追问

- → Partial UNIQUE Index 有什么业务价值?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 046 如何用 Partial UNIQUE Index 保证“每个用户最多一条 active 记录”？

难度 中	频率 高	标签 Partial / Constraint	来源 Partial index

30 秒标准回答

> **结论** 对 user_id 建唯一 partial index，并限定 active=1。

思路分析 / Know-Why

它把跨行业务不变量下沉到数据库：并发两个事务尝试创建第二条 active 记录时，由 UNIQUE 约束原子地拒绝其中一个，比“SELECT 检查后 INSERT”的应用逻辑可靠。

代码 / 白板示例

```
CREATE UNIQUE INDEX one_active
ON subscription(user_id)
WHERE active=1;
```

易错点

- ⚠ 先 SELECT COUNT 再 INSERT，产生竞态。

面试官可能继续追问

- → 如何在 UPSERT 中利用这个唯一索引?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 047 什么是 Expression Index?

难度 中	频率 高	标签 Expression Index	来源 Expression index

30 秒标准回答

> **结论** 对表达式计算结果建立索引，例如 lower(email)、abs(amount)。

思路分析 / Know-Why

适合查询逻辑天然依赖稳定派生表达式的场景。表达式中的函数必须满足 determinism 要求，否则索引中的物化值可能随时间变化而失真。表达式索引与 generated column 都能解决派生字段检索，但抽象层级不同。

代码 / 白板示例

```
CREATE INDEX idx_email_norm ON user(lower(email));
```

易错点

- ⚠ 对 random() 等非确定函数建索引。

面试官可能继续追问

- → Expression Index 与 Generated Column 怎么选?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 048 为什么 INDEX ON (x+y)，查询 WHERE y+x=10 可能不用索引?

难度 中	频率 高	标签 Expression Index	来源 Expression index

30 秒标准回答

> **结论** 因为 SQLite planner 不做一般代数等价推理，表达式通常要与 CREATE INDEX 中写法一致（允许少量语法差异）。

思路分析 / Know-Why

这是非常典型的“数学等价 != planner 可匹配”。工程上应该把常用表达式标准化到同一写法，或者用 generated column 统一抽象，避免 ORM 随机改写表达式顺序。

代码 / 白板示例

```
CREATE INDEX idx_xy ON t(x+y);
-- 更稳：WHERE x+y=10
-- 而不是：WHERE y+x=10
```

易错点

- ⚠ 认为优化器会做交换律证明。

面试官可能继续追问

- → 空白、括号等 minor syntactic difference 是否允许?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 049 为什么 Expression Index 只能使用 deterministic function?

难度 中	频率 高	标签 Deterministic / REINDEX	来源 Expression index / 3.53

30 秒标准回答

> **结论** 索引需要假设同一输入在数据库生命周期内得到稳定结果，否则落盘 index key 会和之后查询计算结果不一致。

思路分析 / Know-Why

SQLite 内建函数并非全部 deterministic；application-defined function 默认也不能直接用于索引，必须在注册时声明合适属性。函数语义升级还可能让旧表达式索引 stale，这正是 3.53 引入 REINDEX EXPRESSIONS 等机制的背景之一。

易错点

- ⚠ 函数实现升级后忘记重建表达式索引。

面试官可能继续追问

- → collation 改变时为什么也可能需要 REINDEX?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 050 为什么“有索引”反而可能比 Full Scan 慢?

难度 中	频率 高	标签 选择性 / Cost	来源 Query planner

30 秒标准回答

> **结论** 当条件选择性很差、命中大量行且索引不覆盖时，index seek 后大量回表随机访问可能比顺序扫表更贵。

思路分析 / Know-Why

例如 status 只有 0/1，查询命中 70% 数据。Planner 的目标不是“使用索引”，而是最低预计成本；ANALYZE 提供统计后可能主动选择 SCAN。还要考虑结果排序、缓存、表宽和 LIMIT。

易错点

- ⚠ 看到 SCAN 就认定优化器错误。

面试官可能继续追问

- → 如何通过 sqlite_stat1 影响估算?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

06

第六章 Query Planner 与 SQL 优化

用 EQP、统计和 cost model 解释“为什么 SQLite 选了这个计划”。

本章目标 掌握本章后，应能在白板上解释关键机制，并能把“结论”推导到“为什么”。回答时优先先给结论，再给底层因果链。

题号	题目	难度/频率
051	**EXPLAIN 与 EXPLAIN QUERY PLAN 有什么区别?**	中 / 高
052	**EQP 中 SCAN 与 SEARCH 有何区别?**	中 / 高
053	**USE TEMP B-TREE FOR ORDER BY 表示什么?**	中 / 高
054	**ANALYZE 为什么会改变执行计划?**	中 / 高
055	**现代 SQLite 为什么推荐 PRAGMA optimize?**	中 / 高
056	**SQLite Join 主要用什么算法?**	中 / 高
057	**SQLite 会严格按 FROM 中的书写顺序 Join 吗?**	中 / 高
058	**OR 条件一定不能走索引吗?**	中 / 高
059	**什么是 Skip-Scan?**	中 / 高
060	**Correlated Subquery 为什么容易慢?**	中 / 高

题 051 EXPLAIN 与 EXPLAIN QUERY PLAN 有什么区别?

难度 中	频率 高	标签 EQP / VDBE	来源 EQP

30 秒标准回答

> **结论** EQP 给出高层访问计划；EXPLAIN 输出更底层的 VDBE bytecode。

思路分析 / Know-Why

调 SQL 性能先看 EQP：SCAN/SEARCH、使用哪个 index、是否 COVERING、是否 TEMP B-TREE、join loop 顺序。只有当要研究执行器细节或确认 opcode 行为时，再下钻 EXPLAIN。输出格式并非永久稳定，不宜让生产程序依赖文本格式。

代码 / 白板示例

```
EXPLAIN QUERY PLAN SELECT ...;
EXPLAIN SELECT ...;
```

易错点

- ⚠ 把 EQP 输出当稳定 API 解析。

面试官可能继续追问

- → 如何看 join 顺序?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 052 EQP 中 SCAN 与 SEARCH 有何区别?

难度 中	频率 高	标签 EQP	来源 EQP

30 秒标准回答

> **结论** SEARCH 表示利用约束只访问一个子集；SCAN 通常访问完整表/索引范围。

思路分析 / Know-Why

注意 “SCAN t USING INDEX idx” 仍可能是完整扫描，只是遍历顺序来自索引。分析时要结合 detail 中的约束 `(a=? AND b>?)`，而不是只盯着“出现 index 字样”。

易错点

- ⚠ 看到 USING INDEX 就认为是高选择性查找。

面试官可能继续追问

- → rowid lookup 在 EQP 中如何显示?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 053 USE TEMP B-TREE FOR ORDER BY 表示什么?

难度 中	频率 高	标签 排序 / EQP	来源 EQP

30 秒标准回答

> **结论** SQLite 需要把中间结果写入临时 B-Tree 排序，因为没有可直接提供目标顺序的访问路径。

思路分析 / Know-Why

如果 ORDER BY 是热点且返回数据大，应尝试设计能同时满足 WHERE 前缀和 ORDER BY 的索引，让 planner 按索引天然顺序输出并配合 LIMIT 早停。也可能出现 GROUP BY/DISTINCT 的 temp b-tree。

易错点

- ⚠ 无脑为每个 ORDER BY 列单独建索引。

面试官可能继续追问

- → 联合索引如何同时服务 filter + order?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 054 ANALYZE 为什么会改变执行计划?

难度 中	频率 高	标签 ANALYZE / 统计	来源 Query planner

30 秒标准回答

> **结论** 它收集数据分布统计，使 planner 能估计不同索引/表访问的行数与成本。

思路分析 / Know-Why

没有统计时 planner 只能使用默认假设，低基数列、高度偏斜分布容易估错。ANALYZE 生成 sqlite_stat* 数据；启用 STAT4 的构建还能保存更丰富的样本信息。统计数据改变后，同一 SQL 选择不同 plan 是正常现象。

易错点

- ⚠ 认为执行计划必须永远固定。

面试官可能继续追问

- → 为什么更新大量数据后应该重新优化统计?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 055 现代 SQLite 为什么推荐 PRAGMA optimize?

难度 中	频率 高	标签 统计 / 运维	来源 Pragma

30 秒标准回答

> **结论** 从 3.46.0 起官方推荐用 PRAGMA optimize 作为运行 ANALYZE 的主要方式，它会按需要、带合理 analysis limit 地更新统计。

思路分析 / Know-Why

短连接可以在关闭前运行 optimize；长连接可在打开时用推荐 mask，并周期执行；CREATE INDEX 等 schema change 后也适合运行。它通常是 no-op 或很快，不代表每次都全库 ANALYZE。

代码 / 白板示例

```
PRAGMA optimize;
```

易错点

- ⚠ 每次启动都全库 ANALYZE 大库。

面试官可能继续追问

- → PRAGMA optimize=0x10002 的意图是什么?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 056 SQLite Join 主要用什么算法?

难度 中	频率 高	标签 Join / Planner	来源 EQP

30 秒标准回答

> **结论** 主要是 nested scans / nested loops；每个 join term 对应一个 loop。

思路分析 / Know-Why

因此 join order 与内层访问路径非常关键。理想模式是外层先把行数压小，内层用索引点查。EQP 的树形缩进就是 loop nesting 顺序。SQLite 会做大量 join reorder 和索引选择优化，不应根据 FROM 书写顺序推断实际顺序。

易错点

- ⚠ 把 SQLite 说成依赖 hash join。

面试官可能继续追问

- → CROSS JOIN 为什么可用于限制 reorder?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 057 SQLite 会严格按 FROM 中的书写顺序 Join 吗?

难度 中	频率 高	标签 Join reorder	来源 Optimizer overview

30 秒标准回答

> **结论** 通常不会，optimizer 会根据成本重新排序 inner joins；某些 join 类型和显式约束会限制 reorder。

思路分析 / Know-Why

面试应强调“SQL 是声明式的”。你写的是结果关系，不是执行脚本。若开发者用 FROM 顺序“手调性能”，很可能在数据分布变化后失效。应靠索引、统计和 EQP 验证。

易错点

- ⚠ 把 SQL 文本顺序当执行顺序。

面试官可能继续追问

- → LEFT JOIN 的可重排空间为什么更小?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 058 OR 条件一定不能走索引吗?

难度 中	频率 高	标签 OR Optimization	来源 EQP/Optimizer

30 秒标准回答

> **结论** 不是。若 OR 各分支都可索引，SQLite 可以使用 MULTI-INDEX OR 等策略分别检索后组合 rowids。

思路分析 / Know-Why

这体现 planner 会把 OR 拆成多个可索引 term 的高级优化。若某个分支不可索引或选择性极差，可能退回其他方案。实际判断应看 EQP，而不是背“OR 导致全表扫描”这种过时规则。

代码 / 白板示例

```
WHERE a=1 OR b=2
```

易错点

- ⚠ 把 OR 一律改写 UNION 而不验证。

面试官可能继续追问

- → OR-to-IN rewrite 什么时候发生?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 059 什么是 Skip-Scan?

难度 中	频率 高	标签 Skip-scan	来源 Optimizer overview

30 秒标准回答

> **结论** 即使联合索引最左列没有 WHERE 条件，若其 distinct 值很少，SQLite 可枚举左列值，再利用后续列做多次范围查找。

思路分析 / Know-Why

例如索引 (role,height)，查询 height>=180；如果 role 只有 student/teacher 两种，planner 可以分别对 role 两个值执行索引范围。它严重依赖统计；没有 ANALYZE 时，planner 很难准确判断重复度是否值得 skip-scan。

易错点

- ⚠ 认为最左前缀规则没有任何例外。

面试官可能继续追问

- → 为什么高基数首列不适合 skip-scan?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 060 Correlated Subquery 为什么容易慢?

难度 中	频率 高	标签 Subquery	来源 EQP

30 秒标准回答

> **结论** 外层每产生一行，内层子查询可能再次执行；若缺索引或外层行数大，成本会成倍增加。

思路分析 / Know-Why

EQP 可能显示 CORRELATED SCALAR SUBQUERY。优化方向包括给关联键建索引、减少 outer rows、改写 EXISTS/JOIN，或让 planner 通过 flatten/co-routine/materialization 选择更合适方案。但“子查询一定比 JOIN 慢”同样是错误绝对化。

易错点

- ⚠ 为了“可读性”忽略 N×inner cost。

面试官可能继续追问

- → 什么是 subquery flattening?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

07

第七章 存储引擎与底层原理

把 SQL 继续向下追到 VDBE、B-Tree、Page、Overflow 与 WITHOUT ROWID。

本章目标 掌握本章后，应能在白板上解释关键机制，并能把“结论”推导到“为什么”。回答时优先先给结论，再给底层因果链。

题号	题目	难度/频率
061	**一条 SQL 在 SQLite 内部大致经历什么?**	中 / 高
062	**VDBE 是什么?**	中 / 高
063	**Rowid Table 的数据如何存?**	中 / 高
064	**Index B-Tree 与 Table B-Tree 有什么差异?**	中 / 高
065	**SQLite Page 是什么?**	中 / 高
066	**为什么大 TEXT/BLOB 可能需要 Overflow Page?**	中 / 高
067	**Varint 是什么？为什么 SQLite 大量使用?**	中 / 高
068	**DELETE 很多数据后文件为什么不立刻变小?**	中 / 高
069	**什么是 WITHOUT ROWID?**	中 / 高
070	**WITHOUT ROWID 有哪些限制和语义变化?**	中 / 高

题 061 一条 SQL 在 SQLite 内部大致经历什么？

难度 中	频率 高	标签 内部架构	来源 Architecture

30 秒标准回答

> **结论** Tokenizer/Parser → Planner/Code Generator → VDBE bytecode → B-Tree → Pager → VFS → OS。

思路分析 / Know-Why

这条路径是全书最重要的“总推理链”。WHERE/ORDER BY 由 planner 选择访问路径；VDBE 执行 opcode；B-Tree 提供逻辑表/索引访问；Pager 把逻辑 page 映射到缓存和事务；VFS 隔离操作系统差异。遇到性能与一致性题，都可以问：问题发生在哪一层？

易错点

- ⚠ 只会 SQL 层，无法解释 IO 与锁。

面试官可能继续追问

- → 哪一层负责 ACID？哪一层理解 index key？

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 062 VDBE 是什么?

难度 中	频率 高	标签 VDBE	来源 Architecture

30 秒标准回答

> **结论** Virtual DataBase Engine，是 SQLite 执行编译后 bytecode 的虚拟机。

思路分析 / Know-Why

sqlite3_prepare_v2() 更像编译阶段，生成 opcode program；sqlite3_step() 推进 program，产生结果行或完成写操作。EXPLAIN 可以看到 OpenRead、Seek、Column、Next、ResultRow 等 opcode，从而把高层 SQL 计划落到执行细节。

易错点

- ⚠ 把 sqlite3_step() 当单纯“取下一行 API”。

面试官可能继续追问

- → prepared statement reuse 为什么提高性能?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 063 Rowid Table 的数据如何存?

难度 中	频率 高	标签 Table B-Tree	来源 File format

30 秒标准回答

> **结论** 普通 SQL table 使用 table b-tree；key 是 64-bit rowid，payload 是按 record format 编码的各列。

思路分析 / Know-Why

因此按 INTEGER PRIMARY KEY/rowid 查找是直接 table b-tree seek。若某列声明 INTEGER PRIMARY KEY，它在 record payload 中无需再保存一份真正值，读取时用 b-tree key 代表它。这个物理结构解释了 rowid alias 的空间和访问优势。

易错点

- ⚠ 认为所有主键都决定物理聚簇顺序。

面试官可能继续追问

- → WITHOUT ROWID 如何不同?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 064 Index B-Tree 与 Table B-Tree 有什么差异?

难度 中	频率 高	标签 Index B-Tree	来源 File format

30 秒标准回答

> **结论** 普通 rowid table：table b-tree 以 rowid 为 key、payload 为整行；secondary index 是 index b-tree，key 由索引列 + rowid 组成。

思路分析 / Know-Why

二级索引 entry 中的 rowid 充当 tie-breaker 和回表指针，所以即使索引列重复，index key 仍能唯一对应表行。WITHOUT ROWID table 的 secondary index 则附带 PRIMARY KEY 作为 row key。

易错点

- ⚠ 说二级索引只存被索引列，不存定位信息。

面试官可能继续追问

- → 为什么复合主键的 WITHOUT ROWID 二级索引可能很宽?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 065 SQLite Page 是什么?

难度 中	频率 高	标签 Page / Pager	来源 File format

30 秒标准回答

> **结论** Pager/文件格式中的固定大小磁盘块，是数据库读写、缓存和 B-Tree 布局的基本单位。

思路分析 / Know-Why

page size 常见 4096，可在允许范围内配置。一次查询不是“读一行一个文件 offset”，而是把相关 page 拉入 page cache。索引 fan-out、overflow、局部性、VACUUM、WAL frames 都以 page 为核心。

易错点

- ⚠ 用 row 作为最小 I/O 单位解释性能。

面试官可能继续追问

- → page size 大小如何影响 fan-out 与大 BLOB?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 066 为什么大 TEXT/BLOB 可能需要 Overflow Page?

难度 中	频率 高	标签 Overflow Page	来源 File format

30 秒标准回答

> **结论** B-Tree cell 不能把任意巨大 payload 全塞在当前 leaf page；超过本地 payload 阈值后，一部分内容通过 overflow page 链保存。

思路分析 / Know-Why

大字段会增加额外 page reads，削弱 locality；如果热点查询只要小列，而表中塞着巨大 BLOB，covering index 或拆表可能减少访问大 payload 的机会。是否把二进制文件放 SQLite 需要根据事务一致性、文件大小和访问模式评估。

易错点

- ⚠ 认为行是连续完整存放在一个 page。

面试官可能继续追问

- → 为什么 SELECT * 可能意外触发更多 overflow I/O?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 067 Varint 是什么？为什么 SQLite 大量使用?

难度 中	频率 高	标签 File format	来源 File format

30 秒标准回答

> **结论** 1-9 字节的变长整数编码，小整数占更少空间。

思路分析 / Know-Why

rowid、payload length、serial type 等结构大量用 varint。对于典型“小数值居多”的数据库，它能显著降低索引/record header 大小，提高一个 page 能容纳的 entry 数，进而提升 B-Tree fan-out 与 cache efficiency。

易错点

- ⚠ 只知道“压缩整数”但解释不了为什么影响树高。

面试官可能继续追问

- → varint 是否适合随机定长寻址?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 068 DELETE 很多数据后文件为什么不立刻变小?

难度 中	频率 高	标签 Freelist	来源 Pragma/File format

30 秒标准回答

> **结论** 释放的 pages 通常进入 freelist，供后续写入重用；文件物理长度不必缩小。

思路分析 / Know-Why

这是一种“逻辑回收 != 物理截断”。可用 PRAGMA freelist_count 查看空闲页；需要真正 compact 时考虑 VACUUM，或预先使用合适的 auto_vacuum 策略。但 VACUUM 是重建操作，有空间和停顿成本。

易错点

- ⚠ 删除 10GB 就期待文件立刻少 10GB。

面试官可能继续追问

- → incremental_vacuum 何时有效?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 069 什么是 WITHOUT ROWID?

难度 中	频率 高	标签 WITHOUT ROWID	来源 File format

30 秒标准回答

> **结论** 让表直接使用 PRIMARY KEY 组成的 index b-tree 存整行，不再维护独立隐藏 rowid table 组织。

思路分析 / Know-Why

对非整数主键或复合主键表，普通 rowid 结构可能同时维护“rowid table + UNIQUE/PK index”，查询主键还需要二次查找；WITHOUT ROWID 可以消除这层重复。代价是主键本身可能更宽，会进入所有 secondary index 的 row key。

代码 / 白板示例

```
CREATE TABLE kv(
  k TEXT PRIMARY KEY,
  v BLOB
) WITHOUT ROWID;
```

易错点

- ⚠ 认为 WITHOUT ROWID 总能省空间。

面试官可能继续追问

- → 什么主键形态最适合 WITHOUT ROWID?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 070 WITHOUT ROWID 有哪些限制和语义变化?

难度 中	频率 高	标签 WITHOUT ROWID	来源 File format

30 秒标准回答

结论 必须有 PRIMARY KEY；INTEGER PRIMARY KEY 不再是 rowid alias；不能使用 AUTOINCREMENT；依赖 rowid 的接口不适用。

思路分析 / Know-Why

last_insert_rowid() 与 incremental blob I/O 等 rowid 相关机制在这种表上没有普通含义。设计前要看应用是否依赖稳定整数 surrogate key；如果大量二级索引都要附带很宽的复合 PK，WITHOUT ROWID 反而可能扩大索引。

易错点

- ⚠ 只为了“新特性”把所有表改 WITHOUT ROWID。

面试官可能继续追问

- → 为什么二级索引必须附带 PRIMARY KEY?

答题方法 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

08

第八章 运维、备份与数据完整性

空间回收、完整性检查、外键、Schema、REINDEX 与 corruption。

本章目标 掌握本章后，应能在白板上解释关键机制，并能把“结论”推导到“为什么”。回答时优先先给结论，再给底层因果链。

题号	题目	难度/频率
071	**VACUUM 到底做什么?**	中 / 高
072	**VACUUM 与 auto_vacuum 有什么区别?**	中 / 高
073	**为什么 VACUUM 后 ROWID 可能改变?**	中 / 高
074	**integrity_check 与 quick_check 有什么区别?**	中 / 高
075	**SQLite Foreign Key 为什么是经典陷阱题?**	中 / 高
076	**如何安全修改 SQLite Schema?**	中 / 高
077	**REINDEX 是什么？为什么 3.53 有 REINDEX EXPRESSIONS?**	中 / 高
078	**什么是 Schema Cookie?**	中 / 高
079	**SQLite 线程安全模式有哪些?**	中 / 高
080	**SQLite 数据库最常见的真正 corruption 原因有哪些?**	中 / 高

题 071 VACUUM 到底做什么?

难度 中	频率 高	标签 VACUUM	来源 VACUUM

30 秒标准回答

> **结论** 重建数据库文件以回收空闲空间、压缩碎片，并可产生更紧凑布局；它不是简单删除 freelist 条目。

思路分析 / Know-Why

VACUUM 通常需要额外临时空间，执行期间会有明显 I/O 与锁影响。它还可能改变没有显式 INTEGER PRIMARY KEY 的 rowid，所以业务绝不能把隐式 rowid 当永久外部 ID。现代应用应基于实际空间回收需求使用，而不是定时盲跑。

易错点

- ⚠ 每天 cron VACUUM 大库而不评估写停顿。

面试官可能继续追问

- → VACUUM INTO 有什么备份价值?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 072 VACUUM 与 auto_vacuum 有什么区别?

难度 中	频率 高	标签 空间回收	来源 Pragma

30 秒标准回答

结论 VACUUM 整体重建；auto_vacuum 通过 pointer-map 等机制在运行中迁移空闲页以收缩文件，但不等同于全库重排。

思路分析 / Know-Why

auto_vacuum=FULL/INCREMENTAL 需要在合适时机启用，且可能增加额外 metadata 与碎片。incremental_vacuum 只有数据库使用 incremental auto_vacuum 且有 freelist page 时才有效。

易错点

- ⚠ 以为 auto_vacuum 能达到 VACUUM 的同样 compact 效果。

面试官可能继续追问

- → 为什么启用 auto_vacuum 可能需要 VACUUM 重建?

答题方法 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 073 为什么 VACUUM 后 ROWID 可能改变?

难度 中	频率 高	标签 ROWID / VACUUM	来源 VACUUM

30 秒标准回答

> **结论** 没有显式 INTEGER PRIMARY KEY 的普通表，其隐式 rowid 不是业务永久 ID，VACUUM 重建时可能重新分配。

思路分析 / Know-Why

这进一步说明应用主键应显式建模。若外部系统、缓存、同步协议持有隐式 rowid，就可能在维护操作后发生静默错配。

易错点

- ⚠ 把 rowid 暴露为云同步 ID。

面试官可能继续追问

- → INTEGER PRIMARY KEY alias 会不会在 VACUUM 中改变?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 074 integrity_check 与 quick_check 有什么区别?

难度 中	频率 高	标签 完整性	来源 Pragma

30 秒标准回答

> **结论** integrity_check 做更完整的低层结构与索引一致性检查；quick_check 跳过部分昂贵检查以更快完成。

思路分析 / Know-Why

integrity_check 检查 B-Tree 顺序、misformatted record、missing/surplus index entries、freelist 等；但它不负责 foreign key errors，FK 要另外运行 PRAGMA foreign_key_check。生产诊断应区分“物理结构完整性”与“关系约束完整性”。

代码 / 白板示例

```
PRAGMA quick_check;
PRAGMA integrity_check;
PRAGMA foreign_key_check;
```

易错点

- ⚠ integrity_check=ok 就认为所有 FK 都正确。

面试官可能继续追问

- → partial integrity_check 有什么边界?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 075 SQLite Foreign Key 为什么是经典陷阱题?

难度 中	频率 高	标签 Foreign Key	来源 Pragma/FK

30 秒标准回答

> **结论** SQLite 支持 FOREIGN KEY，但 enforcement 的默认行为受构建/版本兼容策略影响；工程上应在每个 connection 明确 PRAGMA foreign_keys=ON 并验证。

思路分析 / Know-Why

更重要的是外键性能：DELETE/UPDATE parent 时 SQLite 要检查 child rows，因此 child key 通常需要索引，否则约束检查可能退化为扫描。PRAGMA foreign_key_check 用于发现已有违反。

代码 / 白板示例

```
PRAGMA foreign_keys = ON;
```

易错点

- ⚠ 只在建库连接开一次，连接池新 connection 没开。

面试官可能继续追问

- → 为什么在事务中切换 foreign_keys 可能是 no-op?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 076 如何安全修改 SQLite Schema?

难度 中	频率 高	标签 Migration / 3.53	来源 Release 3.53.4

30 秒标准回答

> **结论** 优先使用当前版本支持的 ALTER TABLE；复杂重构采用“新表 → 复制/转换 → 校验 → 重命名/删除”并放在事务中。

思路分析 / Know-Why

SQLite 3.53 增强了 ALTER TABLE 对部分 NOT NULL/CHECK 约束的添加/移除能力，但应用仍需要考虑旧客户端版本。移动端 migration 最重要的是：逐版本可升级、失败可回滚、schema 与 data migration 一起测试。

易错点

- ⚠ 只测试 latest-1 → latest。

面试官可能继续追问

- → 如何处理旧客户端无法解析新 schema feature?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 077 REINDEX 是什么？为什么 3.53 有 REINDEX EXPRESSIONS?

难度 中	频率 高	标签 REINDEX / 版本	来源 3.53/Pragma

30 秒标准回答

结论 REINDEX 重建索引；REINDEX EXPRESSIONS 专门面向表达式索引，可用于修复函数/表达式语义变化导致的 stale index。

思路分析 / Know-Why

表达式索引保存的是函数输出的物化排序键。如果 deterministic function、collation 或 LIKE 等定义改变，磁盘上旧 index key 可能不再对应当前语义。3.53 还加入 self-healing index 改进，说明“schema 中表达式语义与索引物化一致性”是现实工程问题。

易错点

- ⚠ 修改自定义函数实现后继续相信旧索引。

面试官可能继续追问

- → 为什么 integrity_check 可能报告看似 corruption 的索引问题?

答题方法 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 078 什么是 Schema Cookie?

难度 中	频率 高	标签 Schema / Prepared	来源 File format

30 秒标准回答

> **结论** 数据库 header 中的 schema version/cookie 用于识别 schema 是否变化，从而让 prepared statement 重新编译或返回 schema 相关状态。

思路分析 / Know-Why

Prepared statement 的 bytecode 是针对某个 schema 编译的。ALTER/CREATE/DROP 改变 schema 后，旧计划可能引用过时 root page 或 column layout，因此 SQLite 必须检测并 reprepare。工程上这解释了为什么长期缓存 statement 要正确处理 SQLITE_SCHEMA。

易错点

- ⚠ 认为 prepared statement 可以跨任意 schema migration 永久复用。

面试官可能继续追问

- → sqlite3_prepare_v2 为什么比旧 prepare API 更易自动 reprepare?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 079 SQLite 线程安全模式有哪些?

难度 中	频率 高	标签 线程安全	来源 Threadsafe

30 秒标准回答

> **结论** Single-thread、Multi-thread、Serialized。默认构建通常是 Serialized，但应以实际编译/runtime 配置为准。

思路分析 / Know-Why

Multi-thread 表示 SQLite 全局可被多个线程使用，但同一个 connection/statement 不能并发使用；Serialized 对 connection 访问提供更强 mutex 序列化。线程模式讨论的是 library object thread-safety，不等于数据库层允许多 writer。

易错点

- ⚠ “serialized = 多 writer 串行后就没有 SQLITE_BUSY”。

面试官可能继续追问

- → sqlite3_threadsafe() 能反映 runtime mode 吗?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 080 SQLite 数据库最常见的真正 corruption 原因有哪些?

难度 中	频率 高	标签 Corruption	来源 How to corrupt

30 秒标准回答

> **结论** 底层硬件/文件系统错误、锁实现不可靠、绕过 SQLite 直接修改数据库/WAL、错误备份或删除辅助文件、多个不协调 SQLite 副本等。

思路分析 / Know-Why

SQLite Pager 对正常 crash 很鲁棒，真正危险往往是破坏它的假设。网络文件系统 locking bug 是典型风险。自 3.51 起 SQLite 还针对某些 POSIX close() 导致锁丢失问题增加防护，但应用仍不应绕过 VFS/SQLite 去操作正在使用的文件。

易错点

- ⚠ 看到 corruption 就直接归因“SQLite 不可靠”。

面试官可能继续追问

- → 为什么多个不同 SQLite library 副本可能导致同进程锁协调问题?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

09

第九章 SQLite 高级能力

FTS5、JSONB、Generated Columns、UPSERT：现代 SQLite 已远超“轻量 SQL 文件”。

本章目标 掌握本章后，应能在白板上解释关键机制，并能把“结论”推导到“为什么”。回答时优先先给结论，再给底层因果链。

题号	题目	难度/频率
081	**为什么不能直接用 LIKE '%keyword%' 做大规模全文搜索?**	中 / 高
082	**FTS5 tokenizer 为什么非常重要?**	中 / 高
083	**什么是 FTS5 External Content Table?**	中 / 高
084	**什么是 Contentless FTS5?**	中 / 高
085	**SQLite JSON 是不是独立 column type?**	中 / 高
086	**什么是 SQLite JSONB?**	中 / 高
087	**Generated Column 的 VIRTUAL 与 STORED 有什么区别?**	中 / 高
088	**Generated Column 和 Expression Index 怎么选?**	中 / 高
089	**UPSERT 与 INSERT OR REPLACE 有什么区别?**	中 / 高
090	**SQLite 五种 ON CONFLICT 策略是什么?**	中 / 高

题 081 为什么不能直接用 LIKE '%keyword%' 做大规模全文搜索?

难度 中	频率 高	标签 FTS5	来源 FTS5

30 秒标准回答

> **结论** 前导通配符很难利用普通 B-Tree 前缀顺序，也缺少分词、phrase、ranking 等全文检索能力；应考虑 FTS5。

思路分析 / Know-Why

FTS5 维护 inverted index，把 token 映射到 document/position；可支持 MATCH、phrase、prefix、NEAR、column filter、BM25 等。普通 B-Tree 擅长有序标量定位，全文搜索是完全不同的数据结构问题。

易错点

- ⚠ 给 body 建普通 INDEX 后期待 %keyword% 变快。

面试官可能继续追问

- → trigram tokenizer 什么时候适合 substring 搜索?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 082 FTS5 tokenizer 为什么非常重要?

难度 中	频率 高	标签 FTS5 / Tokenizer	来源 FTS5

30 秒标准回答

> **结论** Tokenizer 决定文本如何拆成可索引 token，因此直接决定召回率、前缀行为、大小写/重音处理和语言适配。

思路分析 / Know-Why

内建 tokenizer 包括 unicode61、ascii、porter、trigram 等。英语 stemming 与 CJK 分词需求不同；若 tokenizer 与用户查询习惯不匹配，再好的 ranking 也无法找回根本没被正确切出的 token。

易错点

- ⚠ 把 tokenizer 当实现细节而不做产品级评估。

面试官可能继续追问

- → unicode61 remove_diacritics 有什么影响?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 083 什么是 FTS5 External Content Table?

难度 中	频率 高	标签 FTS5 / External content	来源 FTS5

30 秒标准回答

> **结论** FTS5 只保存全文索引，原始正文放在另一个普通表/视图；查询需要时由 content table 提供列值。

思路分析 / Know-Why

好处是避免正文重复存储；代价是同步责任落到应用/trigger。如果普通表更新而 FTS index 未同步，查询结果可能不可预测。正确工程通常用 trigger 或明确 rebuild 流程保证一致性。

易错点

- ⚠ 只建 external content FTS 表，不维护更新。

面试官可能继续追问

- → 删除时为什么更新顺序很重要?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 084 什么是 Contentless FTS5?

难度 中	频率 高	标签 FTS5 / 版本	来源 FTS5

30 秒标准回答

> **结论** FTS5 只保存索引、不保留原始列值，能显著节省空间；读取原列通常得到 NULL。

思路分析 / Know-Why

经典 contentless 模式的 UPDATE/DELETE 限制较多。自 3.43 起 contentless-delete 模式改善了常规 UPDATE/DELETE 支持，新代码如果不要求旧版本兼容通常更值得考虑。

易错点

- ⚠ 把 contentless 当普通表使用 SELECT * 取正文。

面试官可能继续追问

- → contentless-delete 的 tombstone/merge 有何含义?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 085 SQLite JSON 是不是独立 column type?

难度 中	频率 高	标签 JSON1	来源 JSON1

30 秒标准回答

> **结论** 通常不是。JSON 文本可以存 TEXT，JSONB 存 BLOB；JSON 函数负责解析、查询和变换。

思路分析 / Know-Why

SQLite 的动态类型系统意味着 schema 仍要自己决定是否用 CHECK(json_valid(...))、generated column 抽取关键字段并索引。若把所有业务字段塞一个 JSON blob，会失去很多关系约束与索引优势。

易错点

- ⚠ 把 JSON 当逃避 schema 设计的万能字段。

面试官可能继续追问

- → 如何给 JSON 中的某个 path 建索引?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 086 什么是 SQLite JSONB?

难度 中	频率 高	标签 JSONB / 版本	来源 JSON1

30 秒标准回答

> **结论** 从 3.45.0 起可把 SQLite 内部 JSON parse-tree representation 以 BLOB 存盘，减少重复解析/渲染开销。

思路分析 / Know-Why

SQLite JSONB 与 PostgreSQL JSONB 仅名称相似，磁盘格式不兼容。它通常比文本 JSON 稍小、处理更省 CPU，但不要假设拥有 PostgreSQL JSONB 的同样 O(1) path lookup 语义。应用应把 JSONB 当 SQLite 私有二进制格式，不自行修改。

易错点

- ⚠ 把 PostgreSQL JSONB blob 直接给 SQLite。

面试官可能继续追问

- → 什么时候 TEXT JSON 更有优势?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 087 Generated Column 的 VIRTUAL 与 STORED 有什么区别?

难度 中	频率 高	标签 Generated Column	来源 Generated columns

30 秒标准回答

> **结论** VIRTUAL 读取时计算、不占实际列存储；STORED 写入时计算并保存，占空间但读时少算。

思路分析 / Know-Why

两者都可参与索引和多种约束。当前 SQLite 不能通过 ALTER TABLE ADD COLUMN 添加 STORED generated column，只能新增 VIRTUAL。选择时看写读比例、表达式计算成本以及兼容版本。

代码 / 白板示例

```
d INT AS (a*abs(b)) VIRTUAL,
e TEXT AS (substr(c,b,b+1)) STORED
```

易错点

- ⚠ 认为 VIRTUAL column 不能建索引。

面试官可能继续追问

- → generated column 的 datatype 由什么决定?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 088 Generated Column 和 Expression Index 怎么选?

难度 中	频率 高	标签 Schema design	来源 Generated/Expression

30 秒标准回答

> **结论** 仅为某个表达式查询加速时 Expression Index 更轻；若派生值需要在多个 SQL 中作为 schema 概念复用，Generated Column 更可维护。

思路分析 / Know-Why

例如 normalized_email 既要查询、唯一约束又要展示调试，generated column 使表达式只定义一次；如果只是 WHERE lower(email)=?，expression index 可能更简单。最终都要关注 deterministic expression 和版本兼容。

易错点

- ⚠ 同时保留等价 generated column 索引和 expression index 造成重复。

面试官可能继续追问

- → VIRTUAL generated column 上的 index 本质是什么?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 089 UPSERT 与 INSERT OR REPLACE 有什么区别?

难度 中	频率 高	标签 UPSERT	来源 UPSERT

30 秒标准回答

> **结论** UPSERT 在唯一冲突时执行 DO UPDATE/DO NOTHING；REPLACE 的冲突处理语义通常是删除冲突旧行再插入新行，副作用完全不同。

思路分析 / Know-Why

REPLACE 可能改变 rowid、触发 DELETE/INSERT 相关行为、影响 FK 和默认值，因此不能把它当“更新现有行”的简写。现代代码如果业务语义是更新，优先使用 ON CONFLICT ... DO UPDATE，并通过 excluded.column 引用待插入值。

代码 / 白板示例

```
INSERT INTO user(id,name) VALUES(1,'Alice')
ON CONFLICT(id) DO UPDATE SET name=excluded.name;
```

易错点

- ⚠ 用 REPLACE 更新父表，意外影响外键/触发器。

面试官可能继续追问

- → UPSERT 的 conflict target 可以是什么?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 090 SQLite 五种 ON CONFLICT 策略是什么?

难度 中	频率 高	标签 Conflict resolution	来源 ON CONFLICT

30 秒标准回答

> **结论** ROLLBACK、ABORT、FAIL、IGNORE、REPLACE。默认通常是 ABORT。

思路分析 / Know-Why

区别在“冲突发生时回滚到哪里、statement 之前已完成的 row changes 是否保留、是否跳过、是否删除旧冲突行”。尤其 ABORT 与 FAIL 常混：ABORT 撤销当前 statement 的变化，FAIL 会停止但保留当前 statement 之前已完成的部分行修改。FK 等某些约束对策略还有特殊行为。

易错点

- ⚠ 把 FAIL 当作 ABORT 同义词。

面试官可能继续追问

- → REPLACE 为什么不是 UPDATE?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

10

第十章 生产故障与系统设计

把前九章知识用于真实场景：锁、批量写、亿级数据、全文检索、迁移和 Offline-First。

本章目标 掌握本章后，应能在白板上解释关键机制，并能把“结论”推导到“为什么”。回答时优先先给结论，再给底层因果链。

题号	题目	难度/频率
091	**线上持续出现 database is locked，如何系统排查?**	中 / 高
092	**100 个线程都要写 SQLite，如何设计?**	中 / 高
093	**插入 100 万条数据为什么“一条一 COMMIT”极慢?**	中 / 高
094	**移动 App：大量读取、少量后台写，你会选择什么 journal mode?**	中 / 高
095	**1 亿行 SQLite 查询慢，第一步做什么?**	中 / 高
096	**如何为 WHERE user_id=? AND status=? ORDER BY created_at DESC LIMIT 20 设计索引?**	中 / 高
097	**一个用户可以有很多历史邮箱，但只能有一个“当前邮箱”，Schema 怎么设计?**	中 / 高
098	**如何设计 1000 万篇文档的本地全文搜索?**	中 / 高
099	**App 跨多个历史版本升级 Schema，migration 怎么设计?**	中 / 高
100	**如何为 Offline-First Notes App 设计 SQLite 架构?**	中 / 高

题 091 线上持续出现 database is locked，如何系统排查?

难度 中	频率 高	标签 故障排查 / S 级	来源 Transaction/WAL

30 秒标准回答

> **结论** 先定位谁持有长事务、是 reader 还是 writer、journal mode、statement 是否未结束，再考虑 timeout；不要第一反应只改 WAL。

思路分析 / Know-Why

排查顺序：1) 记录事务开始/结束与耗时；2) 查是否在事务中做 HTTP/文件/AI 推理；3) 检查 cursor/statement/finalize；4) 检查连接池是否漏 commit；5) 判断 writer 竞争与 read→write upgrade；6) WAL 下看长期 reader/checkpoint starvation；7) 验证文件系统锁可靠。最重要优化通常是“把事务边界缩到只包含数据库操作”。

代码 / 白板示例

```
错误: BEGIN → SELECT → HTTP 2s → UPDATE → COMMIT
正确: HTTP 2s → BEGIN IMMEDIATE → SELECT/UPDATE → COMMIT
```

易错点

- ⚠ 把 busy_timeout 调到 60 秒后宣布修复。

面试官可能继续追问

- → 如何给事务做 observability?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 092 100 个线程都要写 SQLite，如何设计？

难度 中	频率 高	标签 系统设计 / Writer Queue	来源 WAL

30 秒标准回答

> **结论** 不要让 100 个线程无控制争抢同一 writer；常见方案是 producer → queue → dedicated writer，配合 batching 与 backpressure。

思路分析 / Know-Why

这不是因为 SQLite “线程不安全”，而是同一数据库写序列化的物理事实。专用 writer 能合并很多小事务、减少 fsync/journal overhead、让延迟更可控。需要事务独立性的写任务可分批或采用少量受控 writer，但最终仍会在 SQLite writer slot 上串行。

易错点

- ⚠ 为每条请求开 connection 并同时 BEGIN IMMEDIATE。

面试官可能继续追问

- → 何时拆成多个数据库文件能提高并行写？

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 093 插入 100 万条数据为什么“一条一 COMMIT”极慢?

难度 中	频率 高	标签 Bulk insert	来源 Transaction

30 秒标准回答

> **结论** 每条 commit 都产生事务建立、journal/WAL、同步屏障等固定成本；批量放进事务可把固定成本摊薄。

思路分析 / Know-Why

高性能批量写还应：复用 prepared statement、合理 batch size、避免无必要索引/trigger、根据 durability 需求选择 journal/synchronous。若先导入再建索引可减少维护开销，但要考虑在线可用性与临时空间。

代码 / 白板示例

```
BEGIN;
-- reuse prepared INSERT many times
COMMIT;
```

易错点

- ⚠ 为了速度直接 synchronous=OFF 而不评估掉电风险。

面试官可能继续追问

- → batch 过大有什么坏处?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 094 移动 App：大量读取、少量后台写，你会选择什么 journal mode?

难度 中	频率 高	标签 移动端 / WAL	来源 WAL

30 秒标准回答

> **结论** 通常优先测试 WAL，因为它能让 UI reader 与后台 writer 更好并发；但必须同时管理 checkpoint、事务生命周期和 busy handling。

思路分析 / Know-Why

方案不是“一句 WAL 完事”：启动时确认 journal_mode 是否成功；保持 write transaction 短；避免长 cursor；在生命周期合适点 checkpoint；不要把数据库放到不支持 WAL 语义的位置。若应用极少并发访问，rollback mode 也可能更简单。

易错点

- ⚠ 认为 WAL 永远更快、更安全。

面试官可能继续追问

- → 应用进入后台时是否应该 checkpoint?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 095 1 亿行 SQLite 查询慢，第一步做什么？

难度 中	频率 高	标签 性能诊断	来源 EQP/Planner

30 秒标准回答

结论 先 EXPLAIN QUERY PLAN，并测真实返回行数/选择性；再检查索引、排序、统计和 SQL shape，而不是凭行数决定换数据库。

思路分析 / Know-Why

诊断顺序：SCAN/SEARCH？是否使用期望联合/覆盖索引？是否 USE TEMP B-TREE？WHERE 过滤后还剩多少行？LIMIT 能否早停？PRAGMA optimize/统计是否新鲜？表与索引是否过宽？很多“SQLite 太慢”最终是 O(N) query 被错误地执行了。

易错点

- ⚠ 先加所有单列索引，导致写慢但查询仍不理想。

面试官可能继续追问

- → 什么指标说明应该架构升级到 server DB?

答题方法 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 096 如何为 WHERE user_id=? AND status=? ORDER BY created_at DESC LIMIT 20 设计索引?

难度 中	频率 高	标签 索引设计 / S 级	来源 Query planner

30 秒标准回答

> **结论** 典型从等值过滤列开始，再放排序列；若 title 是热点输出且体积可接受，可放尾部形成 covering index。

思路分析 / Know-Why

候选索引 `(user_id,status,created_at DESC,title)` 能让 SQLite 在目标 user/status 的连续区间内按 created_at 顺序读取，取到 20 行即停止；title 使查询可能不回表。是否把 title 加入要 benchmark，因为宽索引会增加写放大与 cache 占用。

代码 / 白板示例

```
CREATE INDEX idx_article_feed
ON article(user_id,status,created_at DESC,title);
```

易错点

- ⚠ 只建 created_at 单列索引，过滤大量无关用户。

面试官可能继续追问

- → status 低基数时把它放第二列是否仍合理?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 097 一个用户可以有很多历史邮箱，但只能有一个“当前邮箱”，Schema 怎么设计？

难度 中	频率 高	标签 约束设计	来源 Partial index

30 秒标准回答

> **结论** 用 Partial UNIQUE Index 把跨行不变量直接交给数据库约束。

思路分析 / Know-Why

`UNIQUE(user_id) WHERE current=1` 可以在并发下原子保证最多一条当前记录。相比应用层 SELECT-then-INSERT，它不会出现两个并发请求都观察到“当前为 0 条”后同时插入的竞态。

代码 / 白板示例

```
CREATE UNIQUE INDEX one_current_email
ON email(user_id)
WHERE current=1;
```

易错点

- ⚠ 只在业务代码里做 if exists 检查。

面试官可能继续追问

- → 切换当前邮箱如何在一个事务中完成?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 098 如何设计 1000 万篇文档的本地全文搜索?

难度 中	频率 高	标签 FTS5 / 系统设计	来源 FTS5

30 秒标准回答

> **结论** 业务 metadata 放普通 table/B-Tree；title/body 用 FTS5；用 rowid/稳定 ID 关联，先全文召回再结合关系过滤和 ranking。

思路分析 / Know-Why

可采用 external content FTS5 避免正文重复；选择合适 tokenizer；用 trigger 保持同步；普通 B-Tree 索引承担 category、time、sync state 等结构化筛选。不要试图用 FTS5 替代所有关系索引，也不要用 LIKE 做全文。

易错点

- ⚠ FTS 索引与业务表不同步。

面试官可能继续追问

- → 如何支持拼写/substring?
- → 如何做 rebuild 与版本迁移?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 099 App 跨多个历史版本升级 Schema，migration 怎么设计?

难度 中	频率 高	标签 Migration	来源 Pragma/Generated

30 秒标准回答

> **结论** 维护单调 schema version，并提供 v1→v2→...→current 的顺序 migration；每步可恢复、可验证、可测试。

思路分析 / Know-Why

移动端用户可能数月不升级，所以必须测试所有受支持起点直达 current，而不仅是 latest-1。复杂 migration 放事务中；升级后运行必要的 constraint/integrity 检查；引入 STRICT、Generated Column 等新 schema 语法前确认旧二进制是否可能重新打开该数据库。

代码 / 白板示例

```
PRAGMA user_version;
```

易错点

- ⚠ 直接 if old_version != current: drop all。

面试官可能继续追问

- → 如何让 migration 具备幂等性?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

题 100 如何为 Offline-First Notes App 设计 SQLite 架构?

难度 中	频率 高	标签 系统设计 / 终极题	来源 WAL/FTS5

30 秒标准回答

> **结论** WAL + 短写事务 + writer serialization + 稳定业务 ID + FTS5 + partial/index + 明确 migration，并把网络同步放在事务外。

思路分析 / Know-Why

本地 notes 表保存 UUID、updated_at、deleted/tombstone、sync_version、dirty 等；`WHERE dirty=1` 可用 partial index；FTS5 为 title/body 提供搜索。同步 worker 先拉取网络结果，再开启短事务合并本地状态；冲突解决（LWW、server revision、CRDT 等）属于同步协议层，而不是 SQLite 自动提供。数据库负责本地原子性，不负责多设备全局一致性。

代码 / 白板示例

```
Remote Sync
   ↓
fetch/resolve (事务外)
   ↓
BEGIN IMMEDIATE
apply changes
COMMIT
   ↓
FTS/index maintained
```

易错点

- ⚠ 拿着 write transaction 等网络响应。
- ⚠ 把 rowid 当跨设备 ID。

面试官可能继续追问

- → 删除同步为何应使用 tombstone 而不是立刻物理 DELETE?

> **答题方法** 先给一句准确结论；再指出适用前提（journal mode / table type / SQLite version）；最后把结论下钻到 Planner、B-Tree、Pager 或文件锁。这样比背 API 更能体现工程理解。

附录 A S 级必刷 25 题

1. 题 012 什么是 Type Affinity?
2. 题 016 STRICT TABLE 解决了什么问题?
3. 题 018 什么是 ROWID?
4. 题 019 INTEGER PRIMARY KEY 和 INT PRIMARY KEY 一样吗?
5. 题 020 INTEGER PRIMARY KEY AUTOINCREMENT 与普通 INTEGER PRIMARY KEY 的区别?
6. 题 022 BEGIN DEFERRED 是什么?
7. 题 023 BEGIN IMMEDIATE 与 DEFERRED 有何区别?
8. 题 024 BEGIN EXCLUSIVE 与 IMMEDIATE 有何区别?
9. 题 026 SQLite 同一数据库能同时有两个 Writer 吗?
10. 题 027 Read transaction 升级为 Write transaction 为什么会失败?
11. 题 032 WAL 的基本原理是什么?
12. 题 033 WAL 为什么允许 Reader 与 Writer 并发?
13. 题 036 Checkpoint 是什么?
14. 题 037 为什么长期 Reader 会导致 WAL 越来越大?
15. 题 042 联合索引 (a,b,c) 能有效服务哪些条件?
16. 题 043 什么是 Covering Index?
17. 题 045 什么是 Partial Index?
18. 题 048 为什么 INDEX ON (x+y)，查询 WHERE y+x=10 可能不用索引?
19. 题 050 为什么“有索引”反而可能比 Full Scan 慢?
20. 题 051 EXPLAIN 与 EXPLAIN QUERY PLAN 有什么区别?
21. 题 054 ANALYZE 为什么会改变执行计划?
22. 题 056 SQLite Join 主要用什么算法?
23. 题 089 UPSERT 与 INSERT OR REPLACE 有什么区别?
24. 题 091 线上持续出现 database is locked，如何系统排查?
25. 题 100 如何为 Offline-First Notes App 设计 SQLite 架构?

> **冲刺建议** 如果只剩 2 小时：先把这 25 题逐题口述到能解释“为什么”；再练题 91、96、100 三道场景题。

附录 B SQLite 心智模型速查

```
SQL
 ↓
Parser / Query Planner
 ↓
VDBE bytecode
 ↓
B-Tree (Table / Index)
 ↓
Pager + Page Cache
 ↓
Journal / WAL + File Locks
 ↓
VFS
 ↓
Operating System / Filesystem
```

并发模型

```
Rollback Journal:
many readers → writer 最终需要更强锁完成提交

WAL:
reader(snapshot) ──┐
reader(snapshot) ──┼── 与 writer 可重叠
writer ────────────┘    但同一时刻仍只有一个 writer
                    ↓
                  WAL
                    ↓ checkpoint
                main database
```

索引设计公式

> **Filter + Order + Cover** 等值过滤列 → 范围/排序列 → 必要的输出列（covering candidate）。先用真实 EQP 与数据分布验证，再决定是否为热点查询增加宽索引。

附录 C 生产排障 Checklist

- database is locked：记录事务开始/结束；找最长 read/write transaction；检查网络 I/O 是否在事务内。
- WAL 过大：找长期 reader；确认 statement/cursor 是否及时结束；检查 checkpoint 策略。
- 查询突然变慢：EXPLAIN QUERY PLAN；检查统计新鲜度；PRAGMA optimize；确认数据分布是否变化。
- 索引没使用：确认表达式/partial predicate 是否匹配；检查选择性、类型/affinity、排序与覆盖。
- 批量写慢：显式 transaction + prepared statement reuse + 合理 batching；评估索引与同步策略。
- 数据库异常：先复制现场；不要随意删除 -wal/-journal；运行 quick_check/integrity_check/foreign_key_check。
- Schema migration：逐版本升级；在真实旧版本数据库上回放；检查 feature compatibility。
- FTS 搜索异常：检查 tokenizer、external content 同步、rebuild/trigger 与版本。

附录 D 官方资料与版本基线

文档	URL	用途
SQLite Release 3.53.4	https://www.sqlite.org/releaselog/3_53_4.html	当前稳定版与 3.53 系列变化。
Transactions	https://www.sqlite.org/lang_transaction.html	DEFERRED / IMMEDIATE / EXCLUSIVE、implicit transaction、读写升级。
Write-Ahead Logging	https://www.sqlite.org/wal.html	WAL、单 writer、checkpoint、共享内存限制。
Database File Format	https://www.sqlite.org/fileformat.html	page、table/index b-tree、rowid、WAL 文件格式。
Datatypes In SQLite	https://www.sqlite.org/datatype3.html	storage class 与 type affinity。
STRICT Tables	https://www.sqlite.org/stricttables.html	STRICT 与 ANY。
Query Planning	https://www.sqlite.org/queryplanner.html	多列索引、covering index 与 cost-based planning。
Indexes On Expressions	https://www.sqlite.org/expridx.html	表达式匹配与 deterministic function。
PRAGMA	https://www.sqlite.org/pragma.html	optimize、integrity_check、foreign_keys、busy_timeout 等。
FTS5	https://www.sqlite.org/fts5.html	tokenizer、external content、contentless。
JSON Functions / JSONB	https://www.sqlite.org/json1.html	JSON1 与 JSONB。
Generated Columns	https://www.sqlite.org/gencol.html	VIRTUAL / STORED。
UPSERT	https://www.sqlite.org/lang_upsert.html	ON CONFLICT DO UPDATE / DO NOTHING。
AUTOINCREMENT	https://www.sqlite.org/autoinc.html	ROWID 自动分配与 sqlite_sequence。
How To Corrupt	https://www.sqlite.org/howtocorrupt.html	文件锁、网络文件系统与 corruption 风险。

—— 完 ——

建议面试前再用 SELECT sqlite_version() 核对目标环境版本。